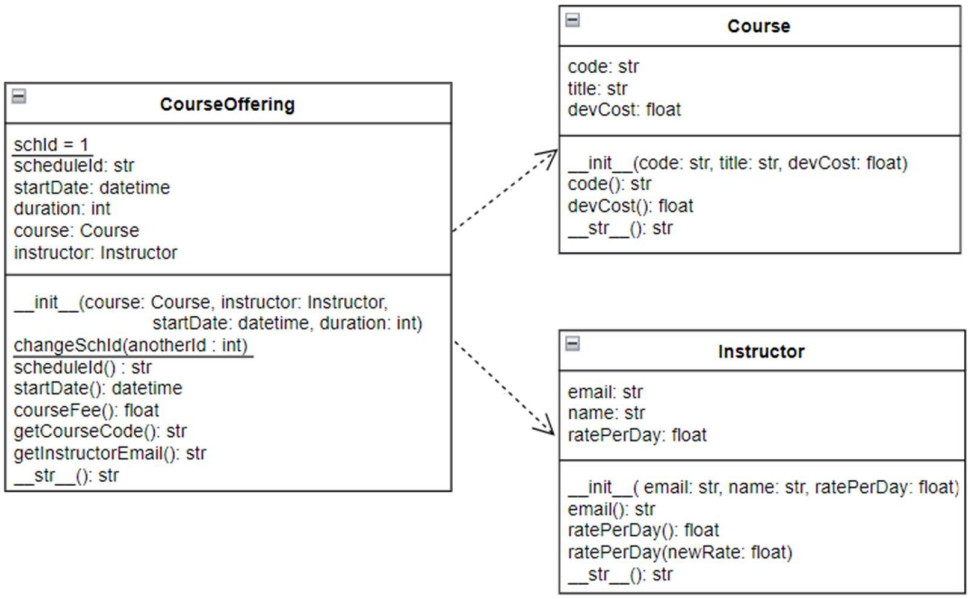
Lab 6 – Sample Exam Paper

1. The following classes model the courses, instructors and course offering by a training provider.



The Course class models a course with code, title and development cost. The Instructor class models an instructor with email, name, and how much the instructor charges per day.

The CourseOffering class models the schedule of a course offered by an instructor, with a start date and duration in days.

Assume that the python code for Course class is already provided.

a) Write the Instructor class according to the class diagram given, with the 3 attributes, a constructor, getter property for email, and getter and setter method for ratePerDay attribute. The __str__() method returns a string in the following format:

```
Instructor email: xxx@yyy.com     Name: Joe Wong
```

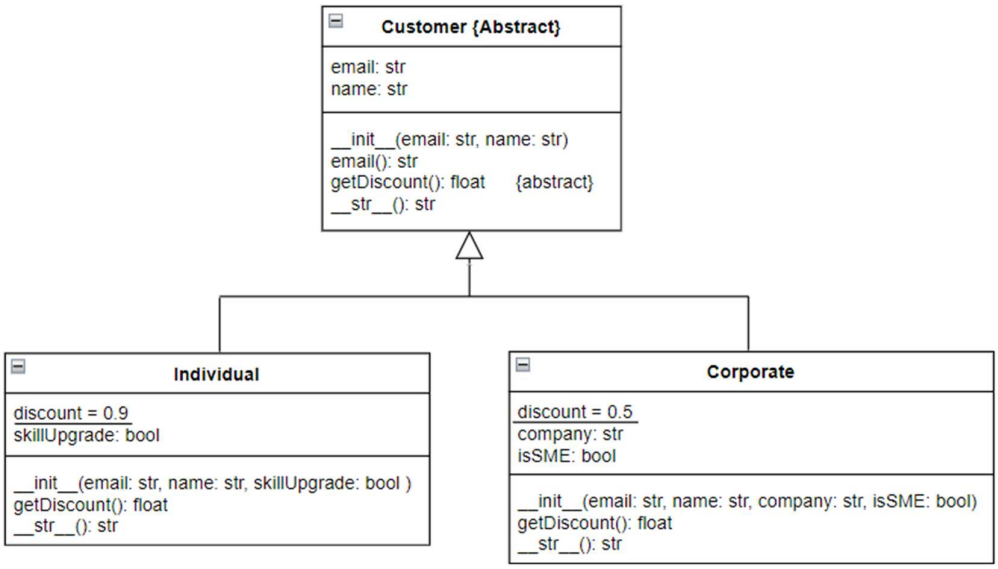
b) Write the CourseOffering class with the following requirements:
   - The scheduleId attribute is obtained by concatenating the course code with the class variable _schId. E.g. if the course code is PY214, then the scheduleId is PY214_1. If there is another course JA201, then the scheduleId is JA201_2 etc. The class variable _schId should increment by 1 for every course schedule object created.
   - The course attribute is the Course object.
   - The instructor attribute is the Instructor object.
   - The datetime attribute is the starting date of the course. It is a datetime object.
   - The duration is an integer value representing the duration in days.
   - Besides the constructor, there are getter methods for scheduleId and startDate.
   - The class method changeSchId(anotherId) modifies the class variable _schId to anotherId in the parameter.
   - The getCourseCode() method returns the course code.
   - The getInstructorEmail() method returns the email of the instructor.
   - The courseFee() method computes and returns the course fee by taking the development cost of the course, plus the duration multiply by the instructor rate per day.

- The __str__() method returns a string in the following format:

```
Schedule Id: PY214_1   Start Date: 1/6/2020   Duration: 3 days
Course Code: PY214 Course            Name: Introduction to Python
Instructor email: xxx@yyy.com     Name: Joe Wong
```

  (the second line of the string comes from the __str__() method of the Course Class)

2. The following classes model customers who make registrations from the training provider.



The Customer class is an abstract class, with the following attributes and methods:

- It has 2 instance variables, email and name
- The constructor initialises the email and name
- Implement a getter property for email
- It has an abstract method which returns the discount the customer is entitled to. It has no implementation.
- str() method which returns a string of the instance variables.

The Individual class is also a Customer, with an additional skillUpgrade instance variable. This value is True if the individual qualifies for skill upgrade. The getDiscount() method returns the discount entitled. Customers who qualify for skillUpgrade enjoys a discount of 90% as indicated in the class variable discount=0.9. Without the skillUpgrade eligibility, there is no discount. The str() method includes the email, name and skillUpgrade status, which should be in this format:

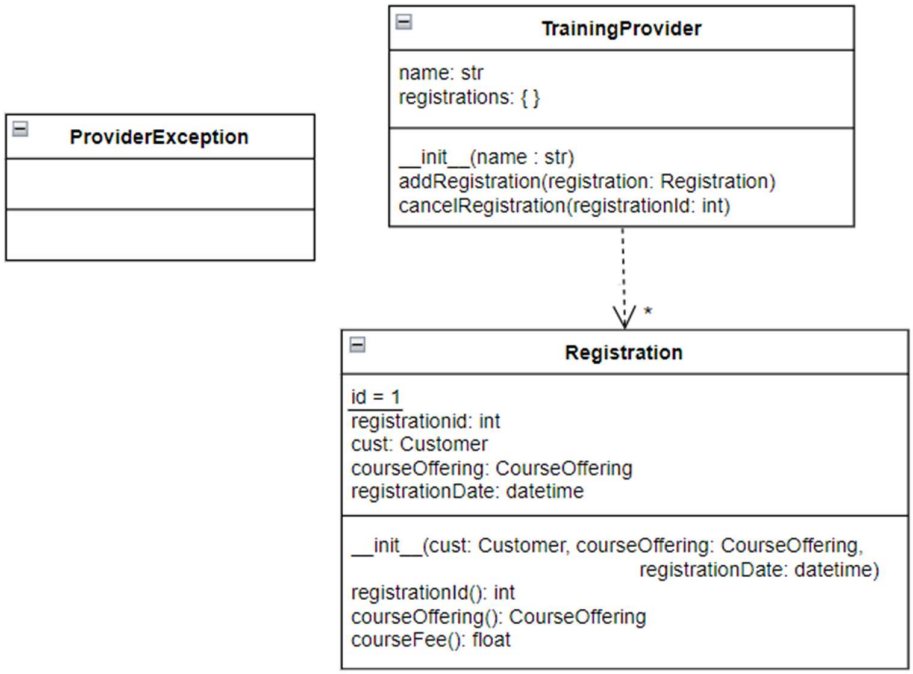
```
Email : joe@email.com   Name: Joe   SkillUpgrade: Yes
                                                (or could be No)
```

The Corporate class is also a Customer. It has additional attributes, company name, and isSME status (True or False). The getDiscount() method returns the discount entitled. All Corporate customers are enjoying 50% discount on course fees as indicated by the class variable discount=0.5, but SMEs get an additional 10% more (i.e 60%). The str() method includes the email, name, company and isSME status as follows:

```
Email : joe@email.com   Name: Joe   Company: SUSS   SME: No
```

Write the Customer and Corporate class.

3. The class diagrams model a Registration that is tracked by the Training Provider.



a) The ProviderException is a subclass of Exception. Write this exception class.

b) The Registration class captures a Customer registering for a CourseOffering. The Registration class has registration id (starts from 1 from the class variable id=1, and increments by 1), customer, courseOffering and registration date as instance variables. It has property for registration id and courseOffering.

In the constructor, if the registration date is later than the courseOffering start date, raise a ProviderException with message “Course offering date cannot be earlier than registration date”. Another consideration is that the registration must be at least 2 days in advance, failing which a ProviderException should be raised, with message “Can only register a course at least 2 days in advance”.

The courseFee() method returns the course fee, which is the course fee from the CourseOffering, taking into consideration the discount the Customer is entitled to.

Write the Registration class.

c) The TrainingProvider class has a collection of registrations. It has the provider’s name as instance variable and a dictionary collection to store registrations. The registration id is the key, and the value is the registration object.

It has the following methods:

- addRegistration(registration)
  The method adds a Registration object to the dictionary. If the customer has already registered for the same course offering, raise a ProviderException with the message "Customer has already registered for this course!"

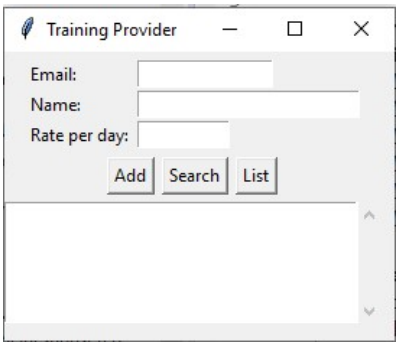
- cancelRegistration(registrationId)
  The method removes a registration from the dictionary given the registration id. If the registration id is not found, raise a ProviderException with the message "Registration id not found!". If the course has started, i.e. the start date of the course is on the same day as the cancellation date or earlier, the registration cannot be cancelled. Raise a ProviderException with a message.

Write the TrainingProvider class.

d) Write a main() function as follows:
   - Create a TraingingProvider tp, for 'SUSS'
   - Create a Course c1, for 'Prog101', 'Python Programming', $1000.00
   - Create an Instructor inst, 'John@email.com', 'John', $200.00
   - Create a CourseOffering co, for 'Prog101', by 'John', 1/10/2024, 2 days
   - Create a CorporateCustomer cc with company which is a SME
   - Create a Registration for r1 for the corporateCustomer
   - Add the registration to the Training provider.

Ensure that all exceptions are caught.

4. The following GUI is used to add/search/List instructors with a training provider:



The code for the GUI is provided, but the event handlers for the Add/Search/List buttons are not. You are NOT required to rewrite the whole program again but indicate only the statement number which needs modification and also the statement number where code should be inserted.

A dictionary to store instructors is provided in line 7. For Add button, create an Instructor object based on the 3 text fields and add the object to the dictionary. The key is the email, and the value the Instructor object. There is no need to consider blank fields, but a message "Instructor already added!" should be displayed if another instructor of the same email is added. Display "Instructor added!" if the instructor is added successfully to the dictionary.

For Search button, retrieve the Instructor object based on the email entered in the email text field. Display the details in the scrolled text using the __str__ of the instructor object. If no instructor is found, display “No instructor with this emai!”.

The List button simply displays all the instructors in the scrolled text, one instructor per line. If the dictionary is empty, display “No instructors”.

```
import tkinter as tk
from tkinter import ttk
from tkinter import scrolledtext

class TrainingProviderGUI():
    def __init__(self):
        self.__instructors = {}
        self.__tk = tk.Tk()
        self.__tk.title("Training Provider")
        self.createWidgets()
        self.__tk.mainloop()

    def createWidgets(self):
        self.__lblEmail = ttk.Label(text='Email: ')
        self.__tbxEmail = ttk.Entry(width=15)
        self.__lblEmail.grid(row=0, column=0, sticky='W')
        self.__tbxEmail.grid(row=0, column=1, sticky='W')

        self.__lblName = ttk.Label(text='Name: ')
        self.__tbxName = ttk.Entry(width=25)
        self.__lblName.grid(row=1, column=0, sticky='W')
        self.__tbxName.grid(row=1, column=1, sticky='W')

        self.__lblRate = ttk.Label(text='Rate per day: ')
        self.__tbxRate = ttk.Entry(width=10)
        self.__lblRate.grid(row=2, column=0, sticky='W')
        self.__tbxRate.grid(row=2, column=1, sticky='W')

        buttonFrame = ttk.Frame(self.__tk)
        self.__btnSearch = ttk.Button(buttonFrame, text='Add')
        self.__btnModify = ttk.Button(buttonFrame, text='Search')
        self.__btnList = ttk.Button(buttonFrame, text='List')
        self.__btnSearch.grid(row=0, column=0)
        self.__btnModify.grid(row=0, column=1, padx=5)
        self.__btnList.grid(row=0, column=2)
        buttonFrame.grid(row=3, column=0, columnspan=2 )

        self.__sclText = scrolledtext.ScrolledText(width=30, height=5, wrap=tk.WORD)
        self.__sclText.grid(row=4, column=0, columnspan=2, pady=5)
```

~ end ~